UNIVERSIDAD PRIVADA "FRANZ TAMAYO"

FACULTAD DE INGENIERIA
CARRERA INGENIERIA DE SISTEMAS



PRÁCTICA DE GIT – CONTROL DE VERSIONES

ESTUDIANTE:
ANTONIO SANCHEZ GUTIERREZ

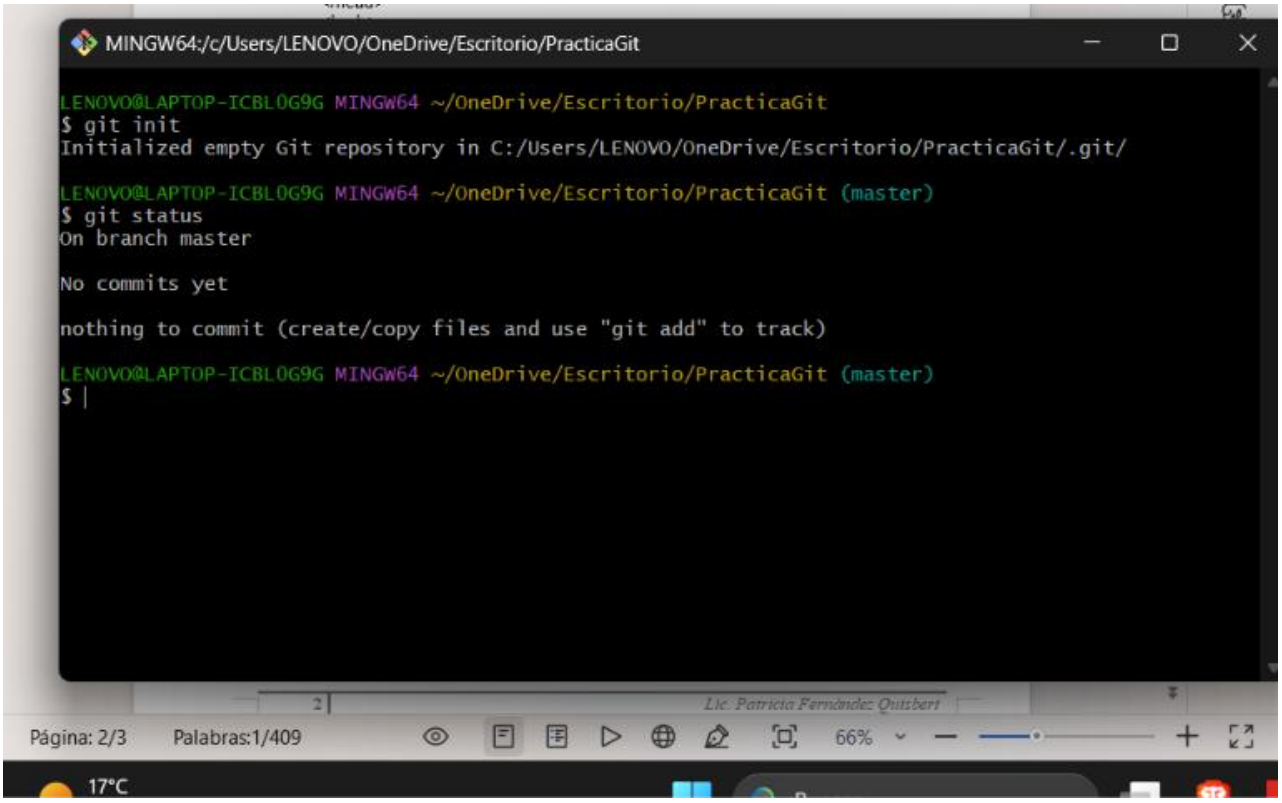
DOCENTE:
ING. PATRICIA FERNANDEZ QUISBERT

EL ALTO-BOLIVIA

A. Crear el repositorio

1. Cree una carpeta llamada: PracticaGit
2. Ingrese a la carpeta.
3. Inicialice un repositorio Git.
4. Verifique el estado del repositorio.

EVIDENCIA:

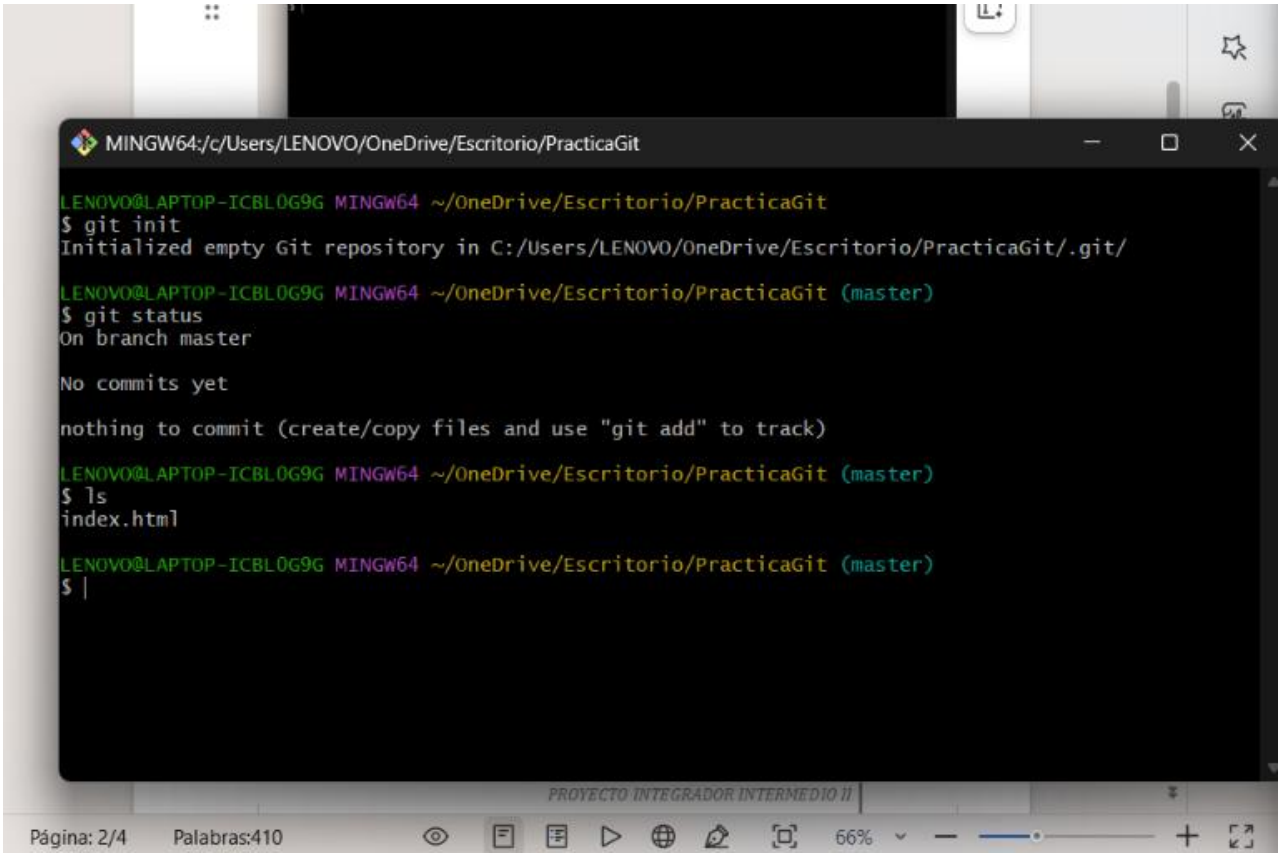


B. Área de trabajo y primer commit

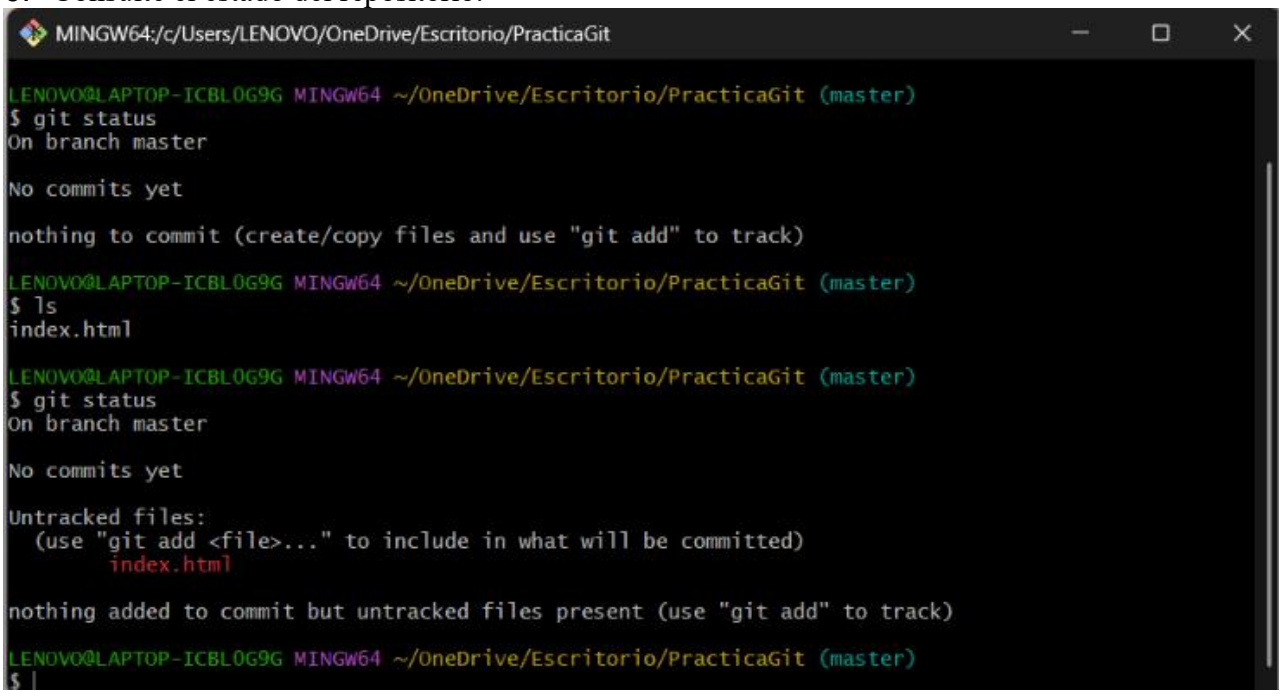
5. Cree un archivo llamado: **index.html** con el siguiente contenido:

```
<!DOCTYPE html>
<html>
<head>
   <title>Mi Perfil</title>
</head>
<body>
   <h1> Ingeniería de Sistemas </h1>
   <p>Estudiante: Su nombre</p>
</body>
</html>
```

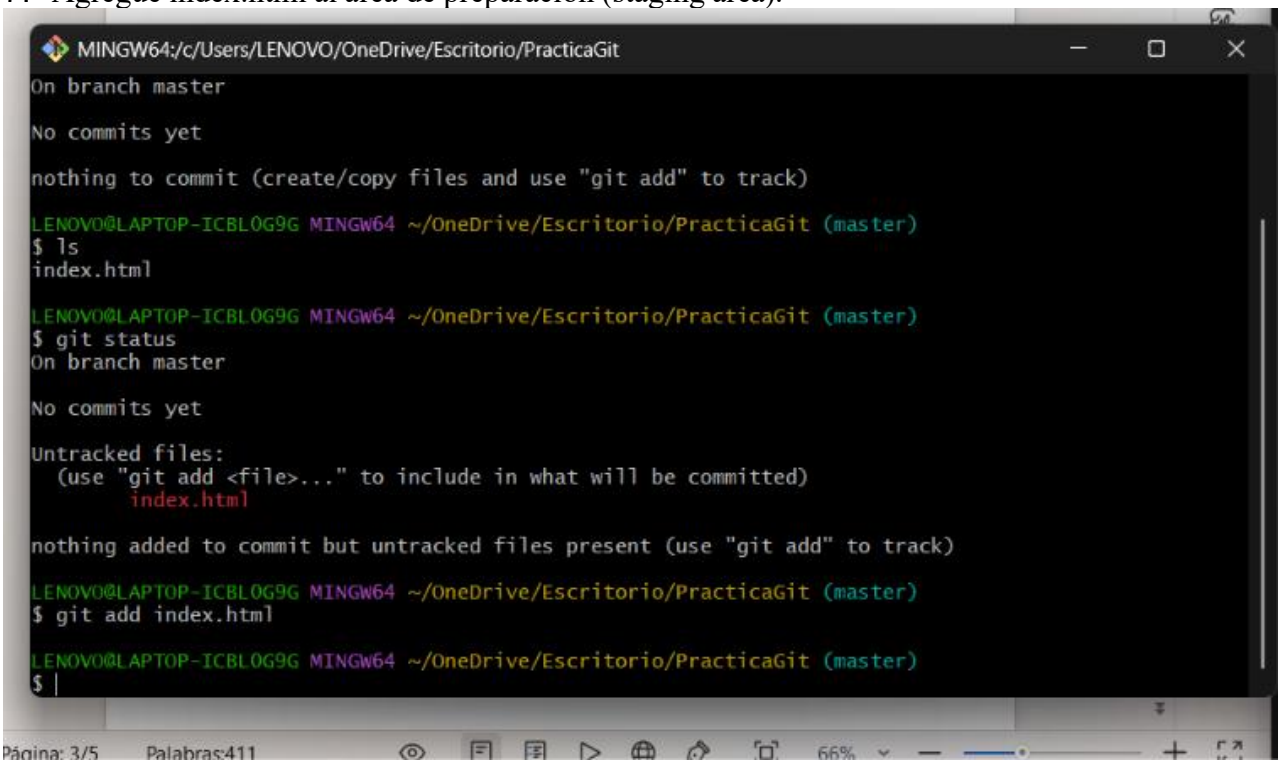
EVIDENCIA:



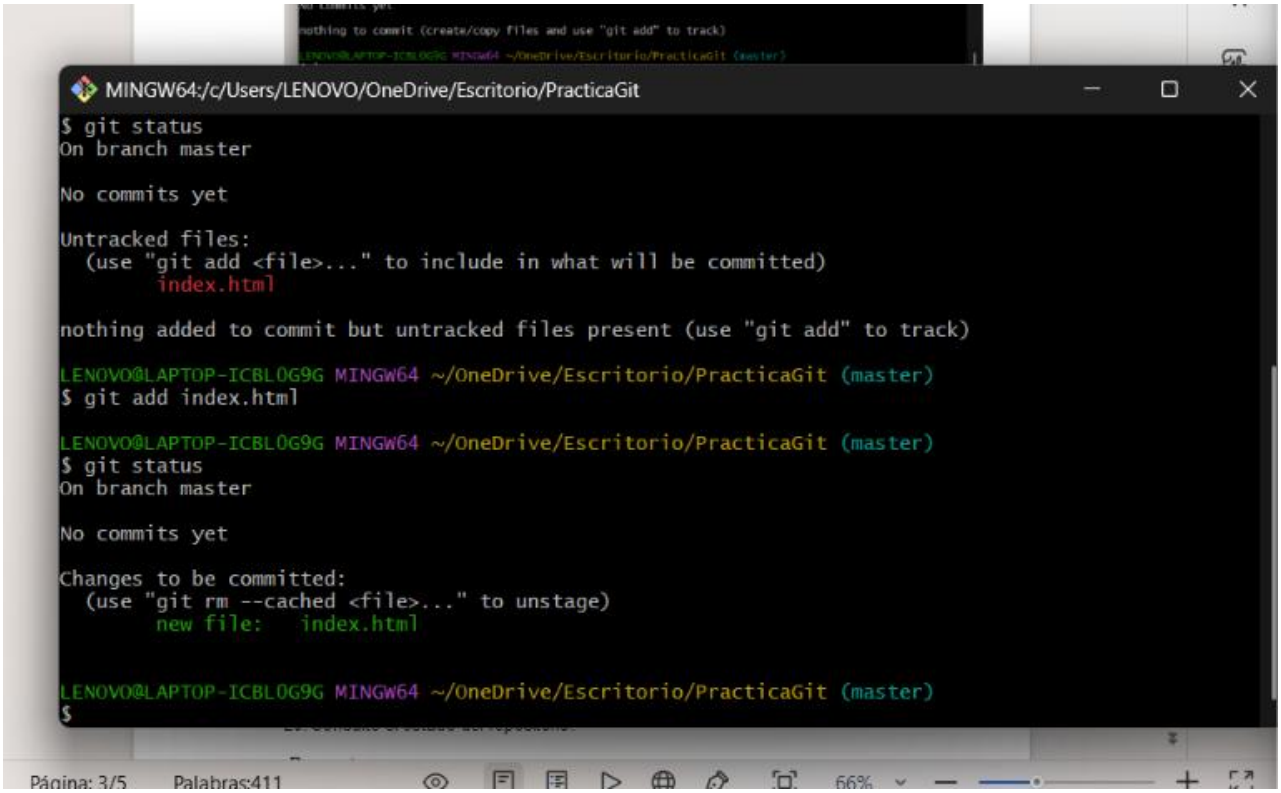
6. Consulte el estado del repositorio.



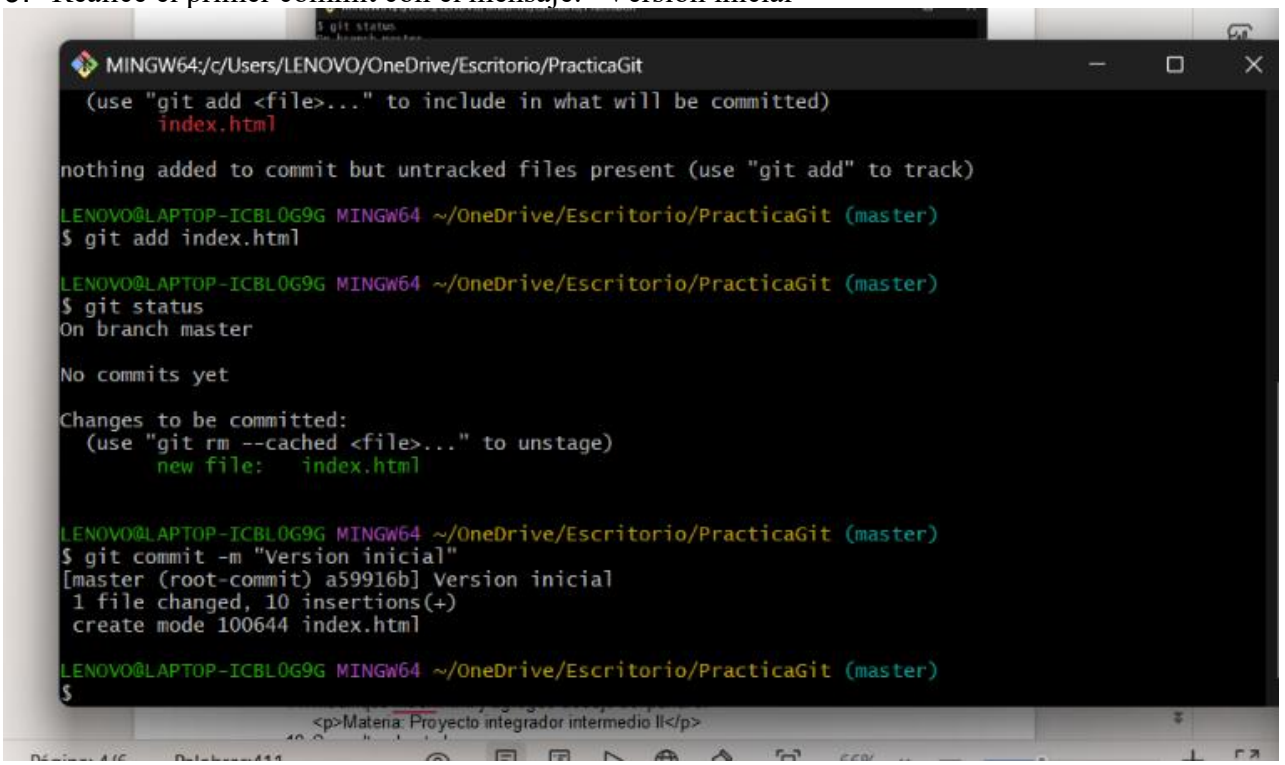
7. Agregue index.html al área de preparación (staging area).



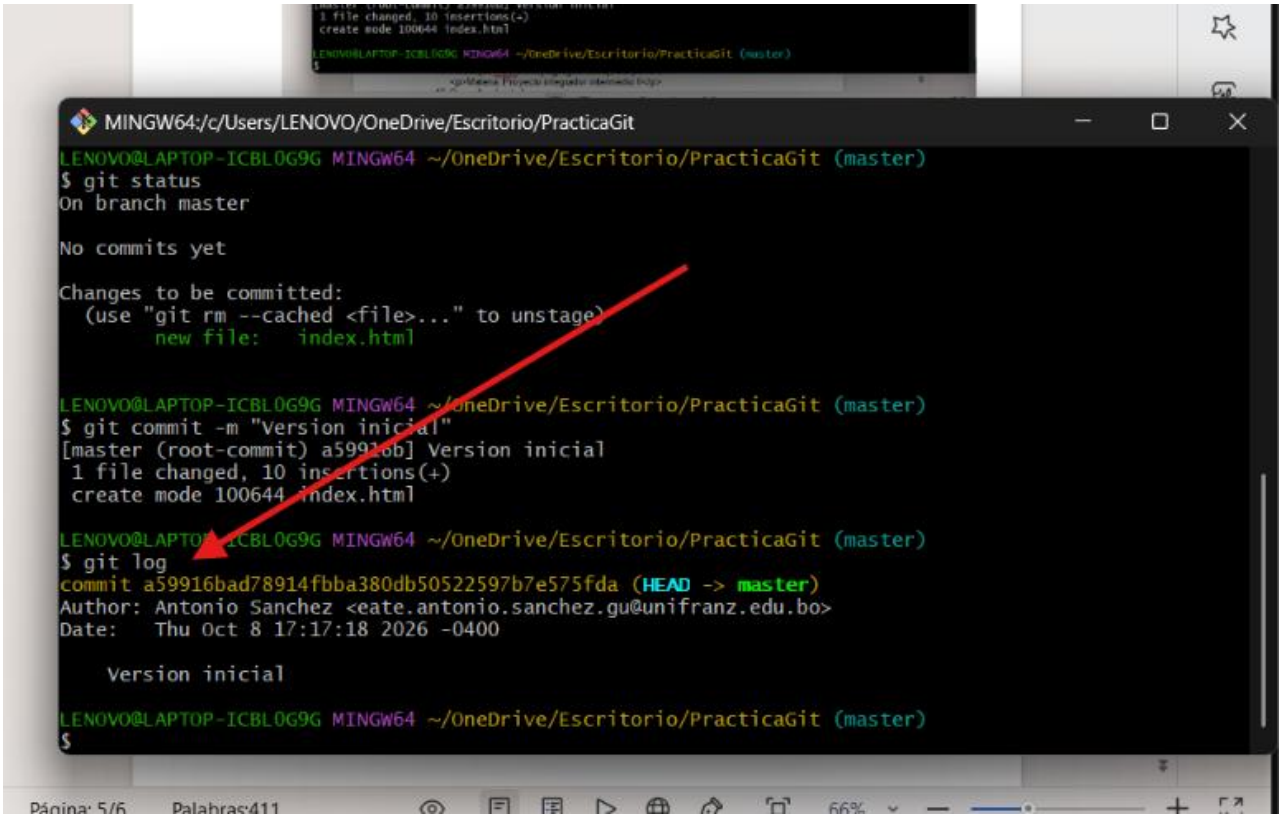
8. Consulte nuevamente el estado.



9. Realice el primer commit con el mensaje: “Version inicial”

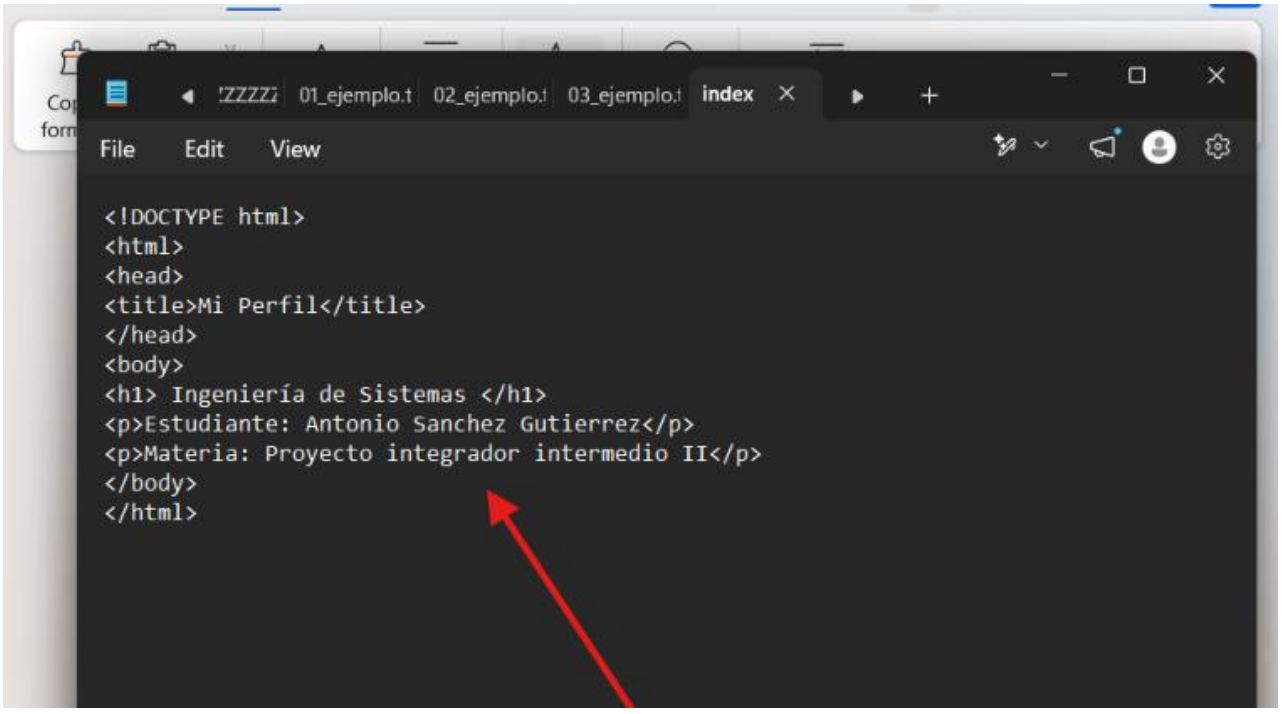


10. Consulte el historial.

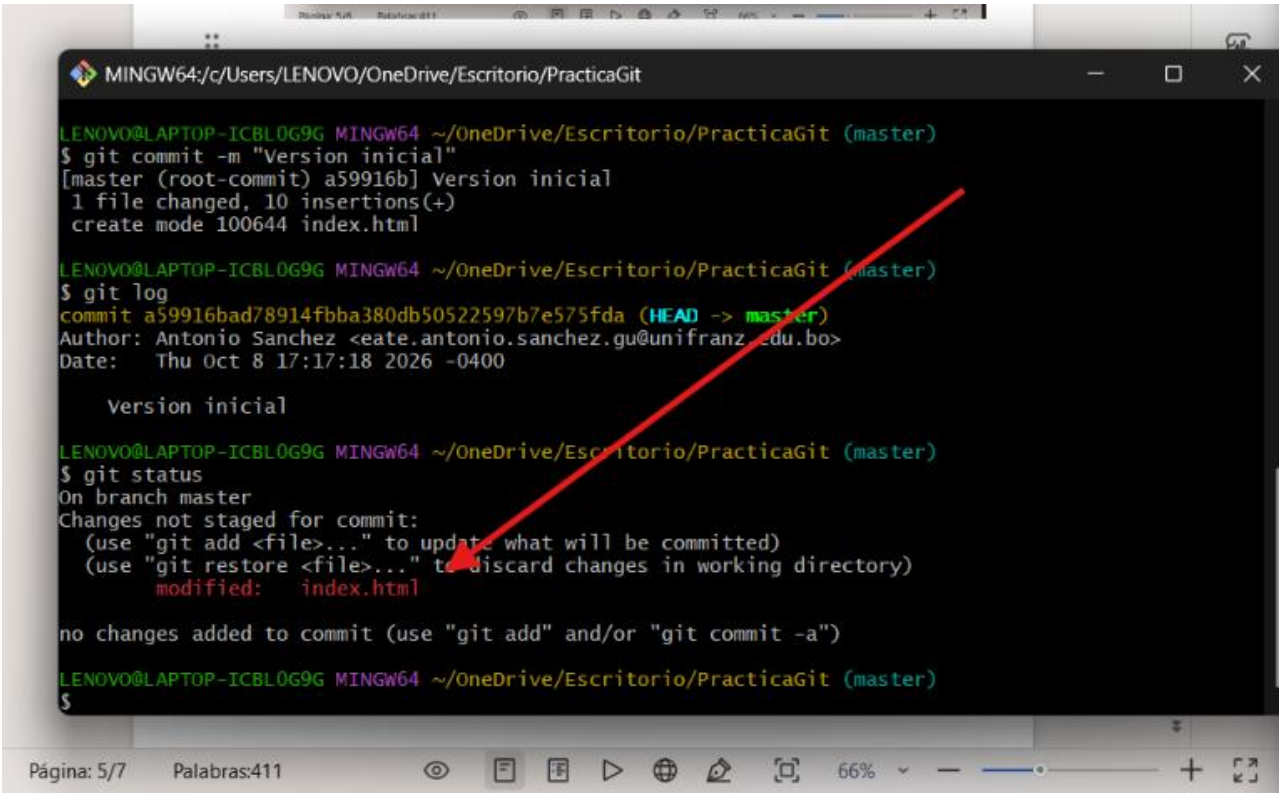


C. Modificar y guardar una nueva versión

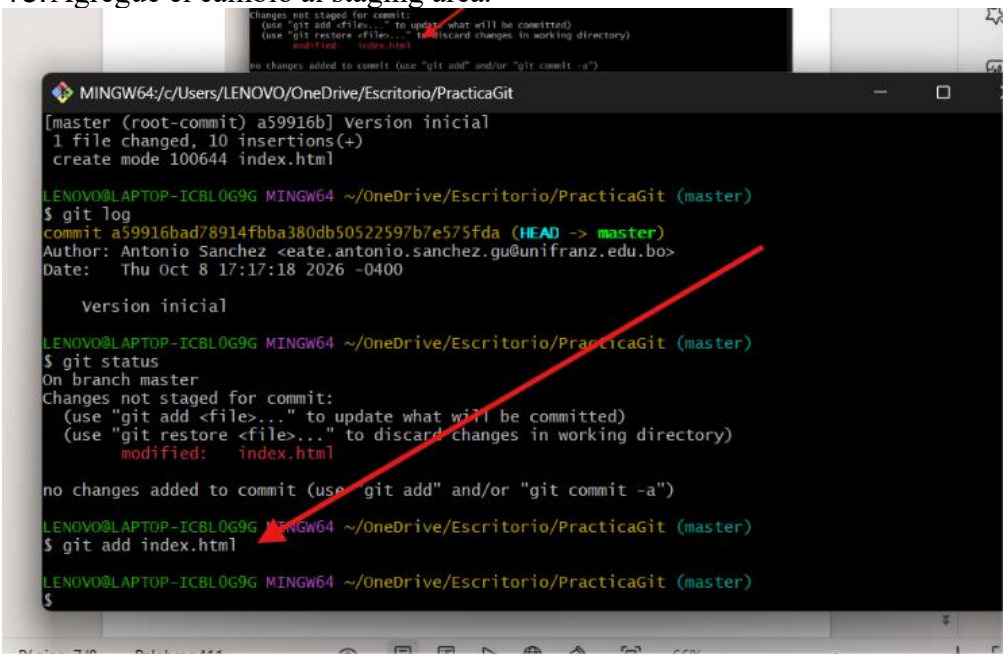
11. Modifique index.html y agregue debajo del párrafo:
    Materia: Proyecto integrador intermedio II



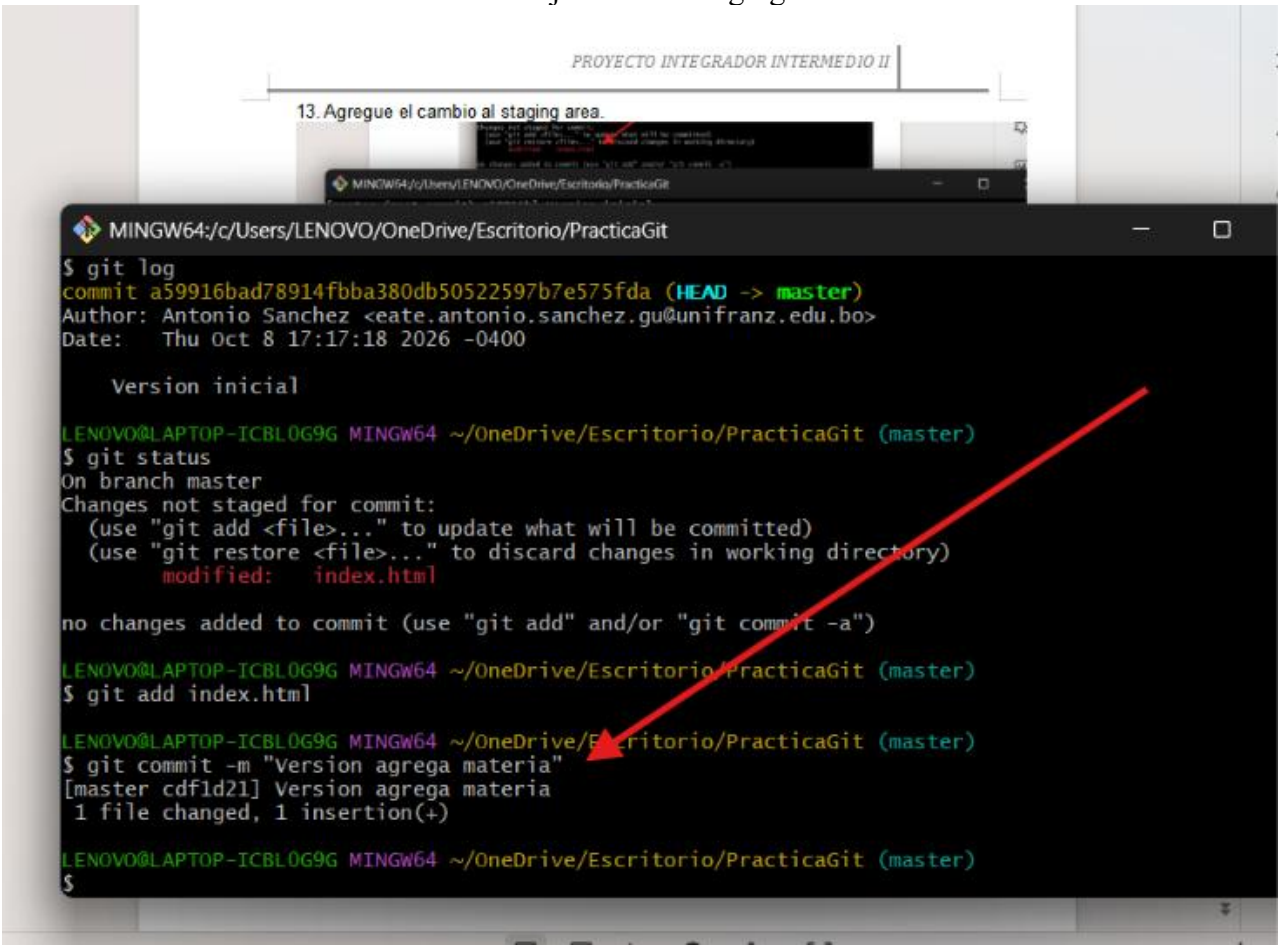
12. Consulte el estado.



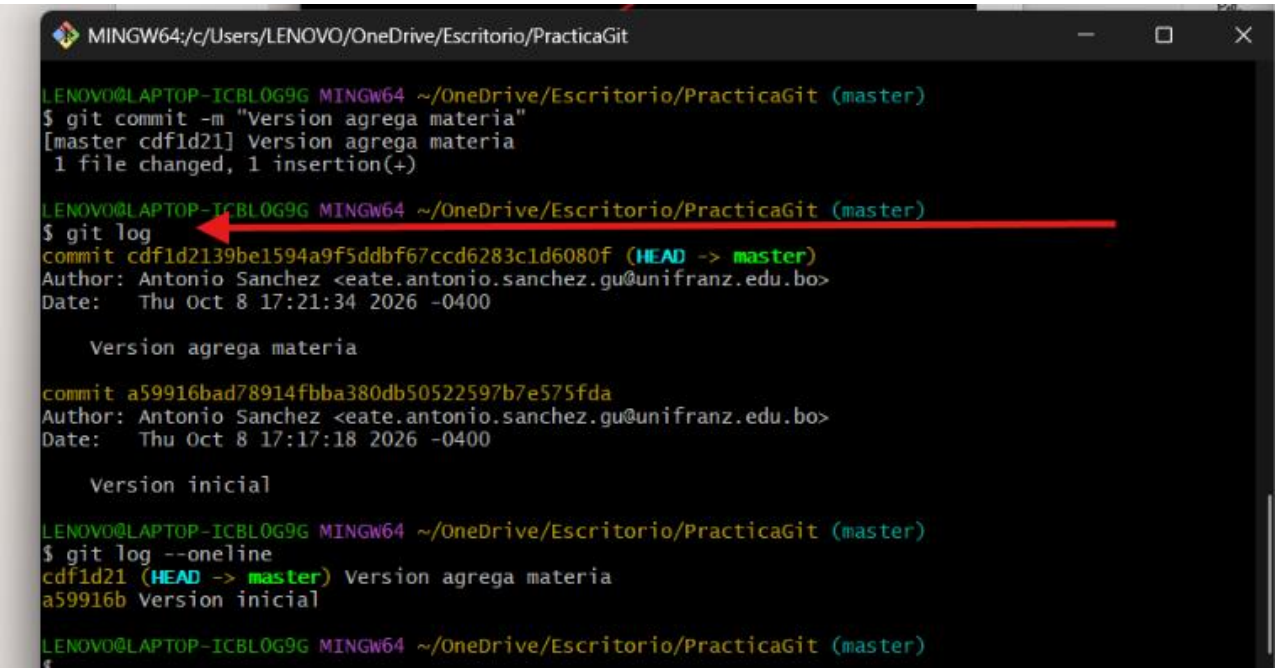
13. Agregue el cambio al staging area.



14. Realice un nuevo commit con el mensaje: “Version agrega materia”

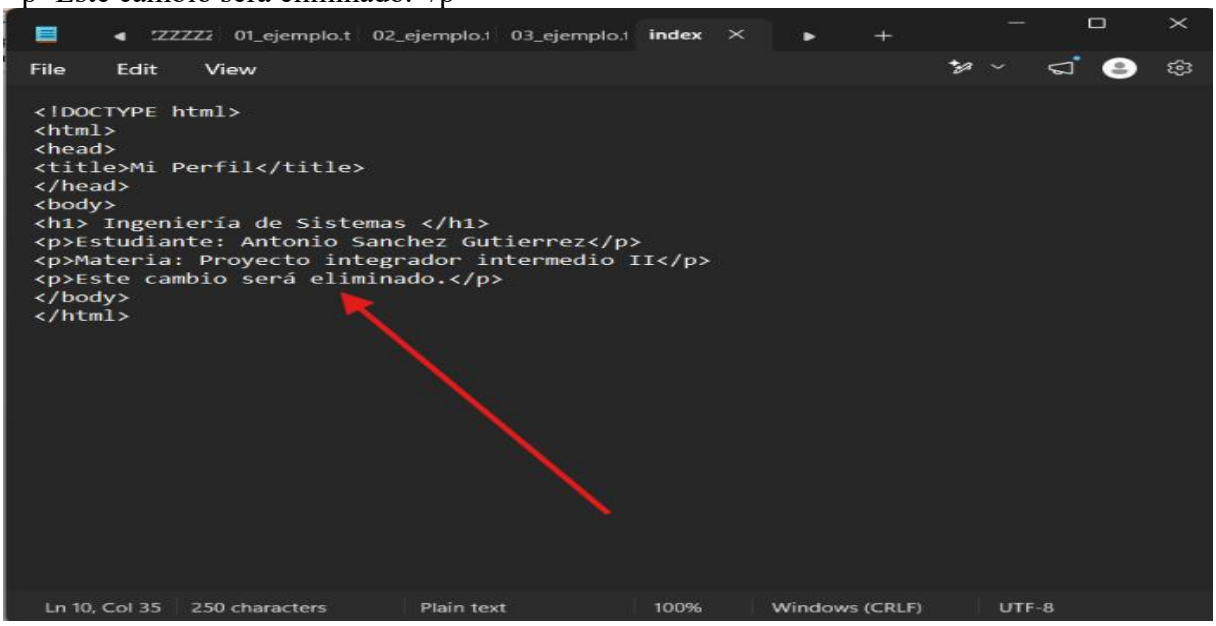


15. Consulte el historial.

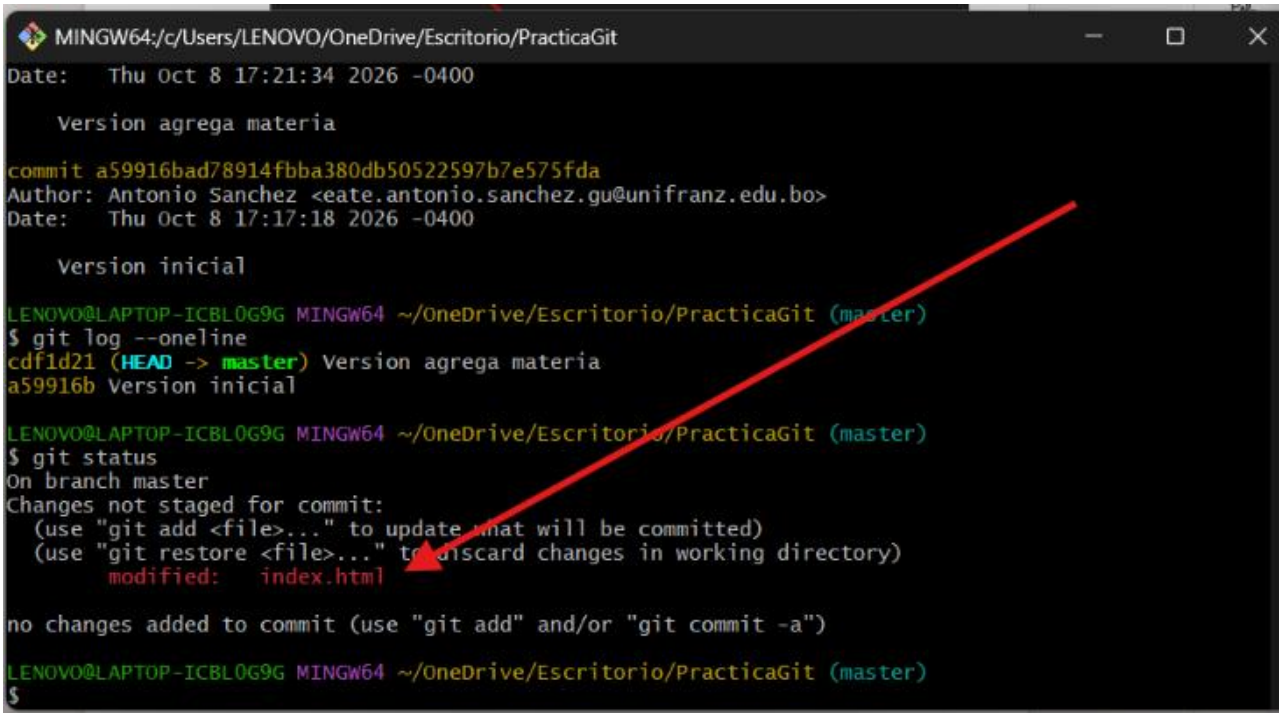


D. Practicar restore

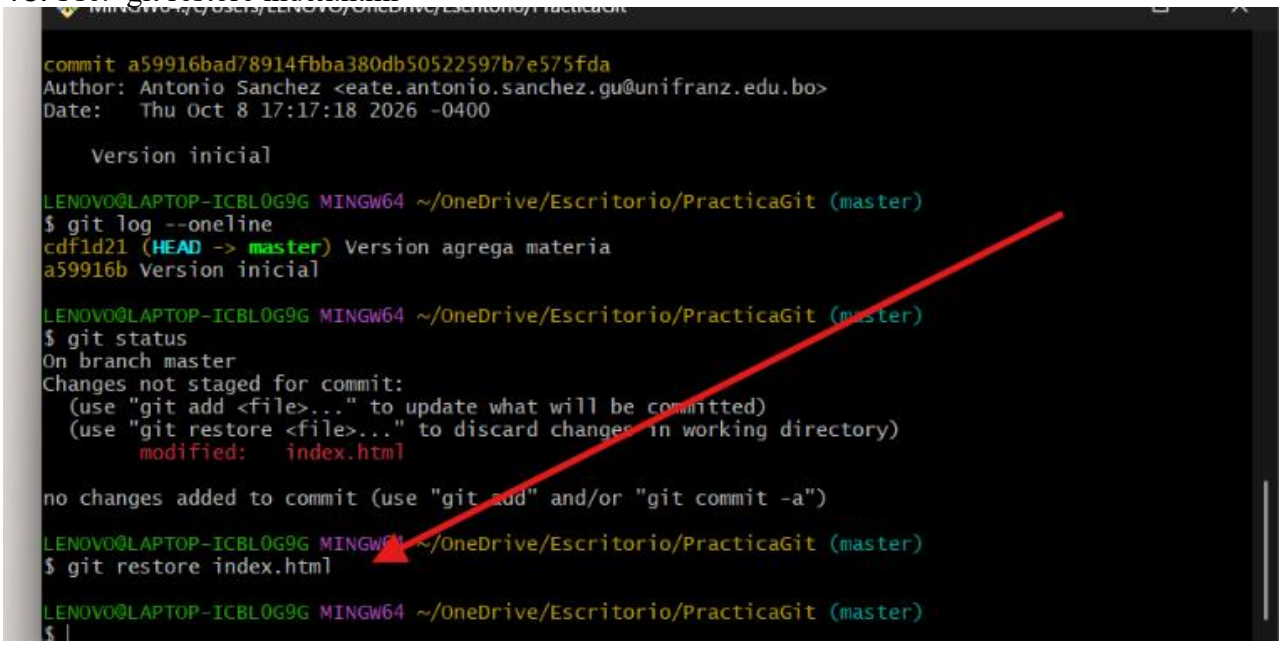
16. Modifique nuevamente index.html agregando:
Este cambio será eliminado.



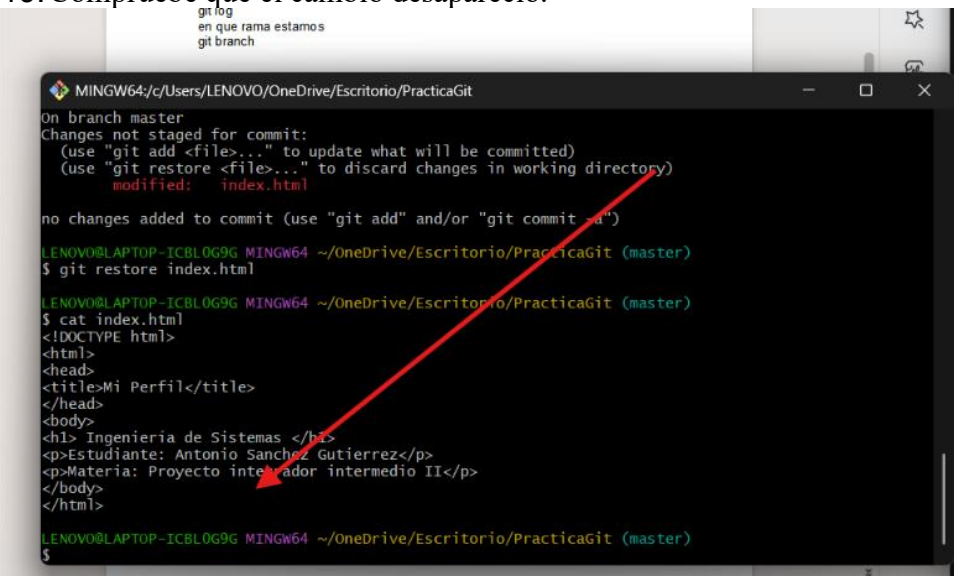
17. Consulte el estado.



18. Use: git restore index.html

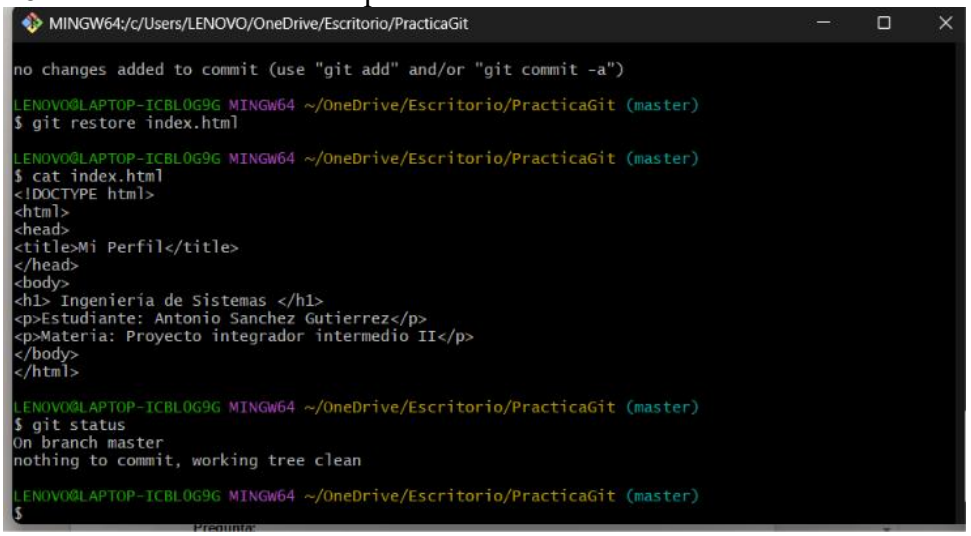


19. Compruebe que el cambio desapareció.



Observamos que desaparecio la linea que agregamos

20. Consulte el estado del repositorio.



Pregunta:

¿Cómo recupera este archivo? Origen, Destino

Seleccione entre: área de trabajo, área de preparación, repositorio

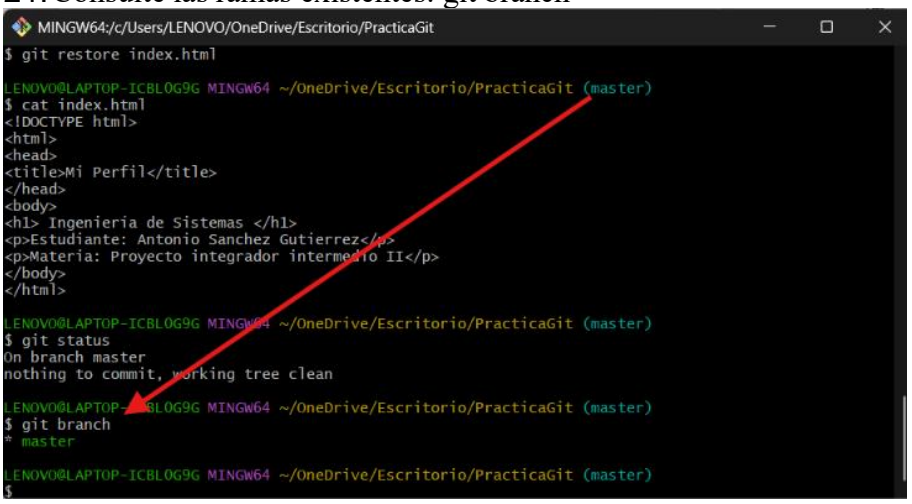
Origen: área de preparación (que en este momento es igual al último commit guardado en el repositorio).

Destino: área de trabajo.

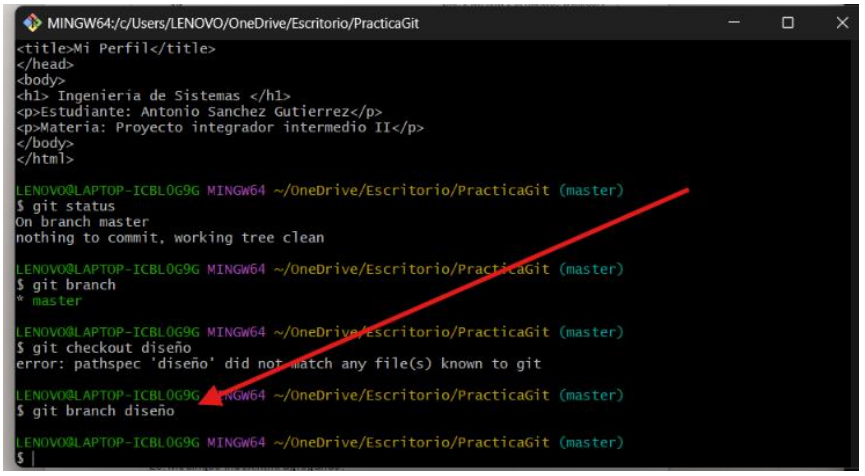
Basicamnete git restore toma la version guardada y la copia de vuelta a tu carpeta de trabajo, borrando el cambio que no habiamos guardado.

E. Crear y utilizar una rama

21. Consulte las ramas existentes: git branch

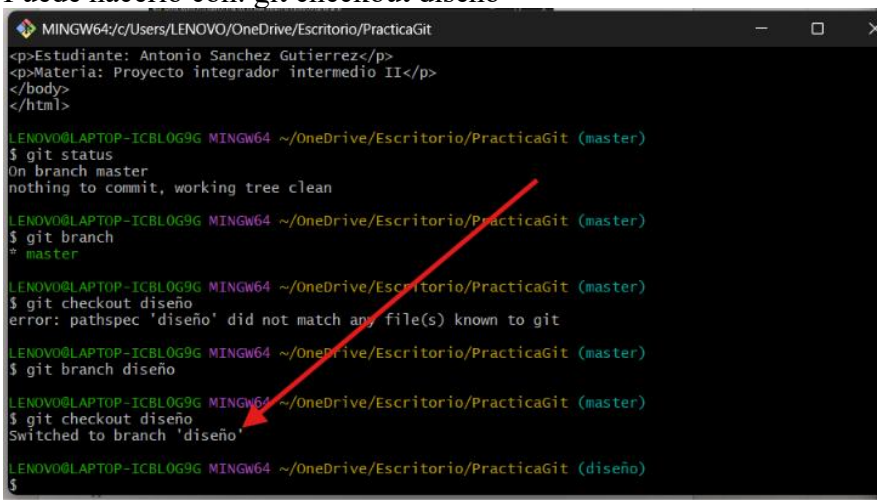


22. Cree una rama llamada: diseño

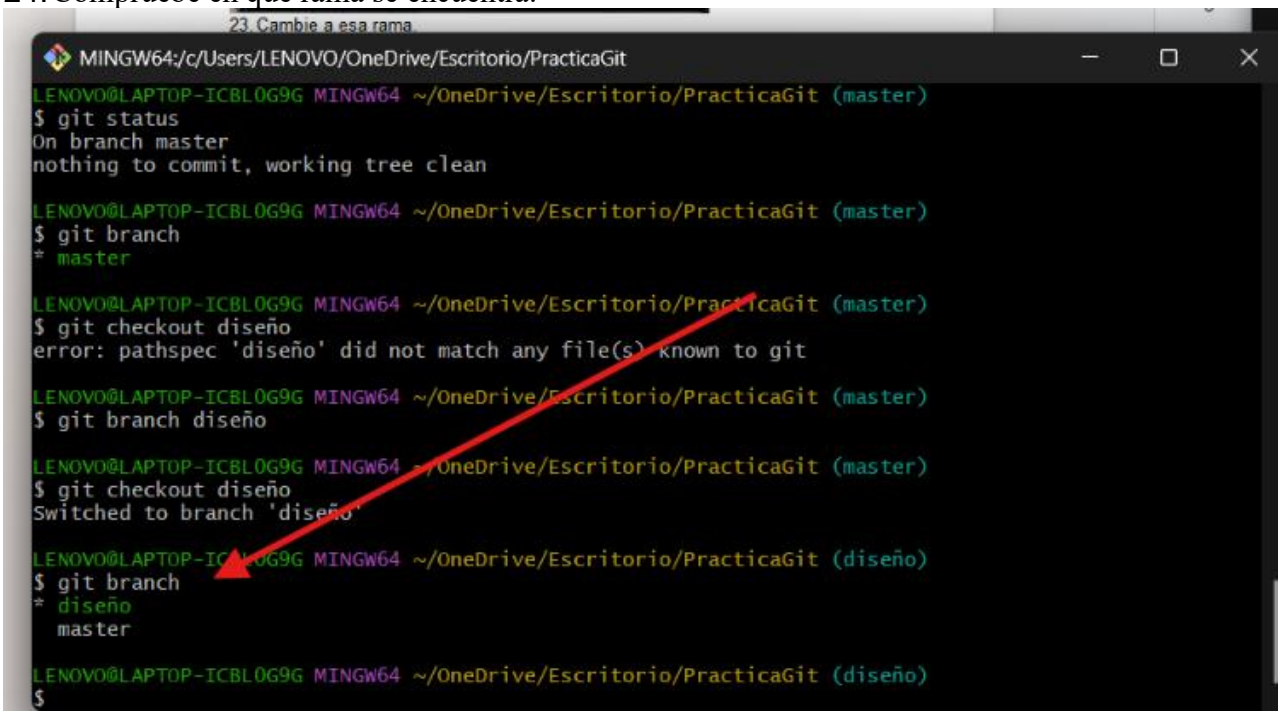


23. Cambie a esa rama.

Puede hacerlo con: git checkout diseño

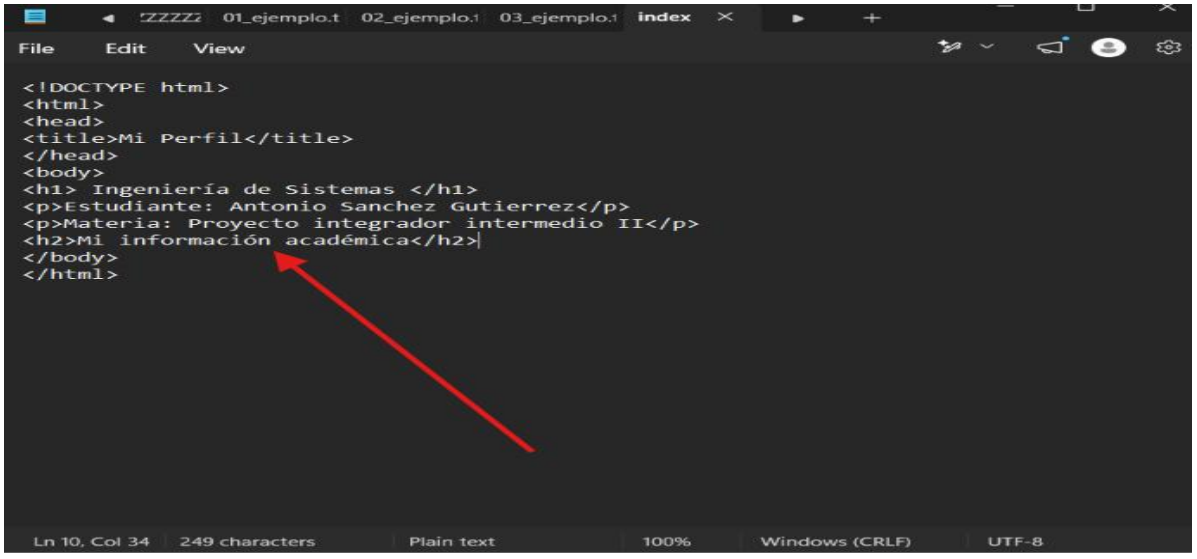


24. Compruebe en qué rama se encuentra.

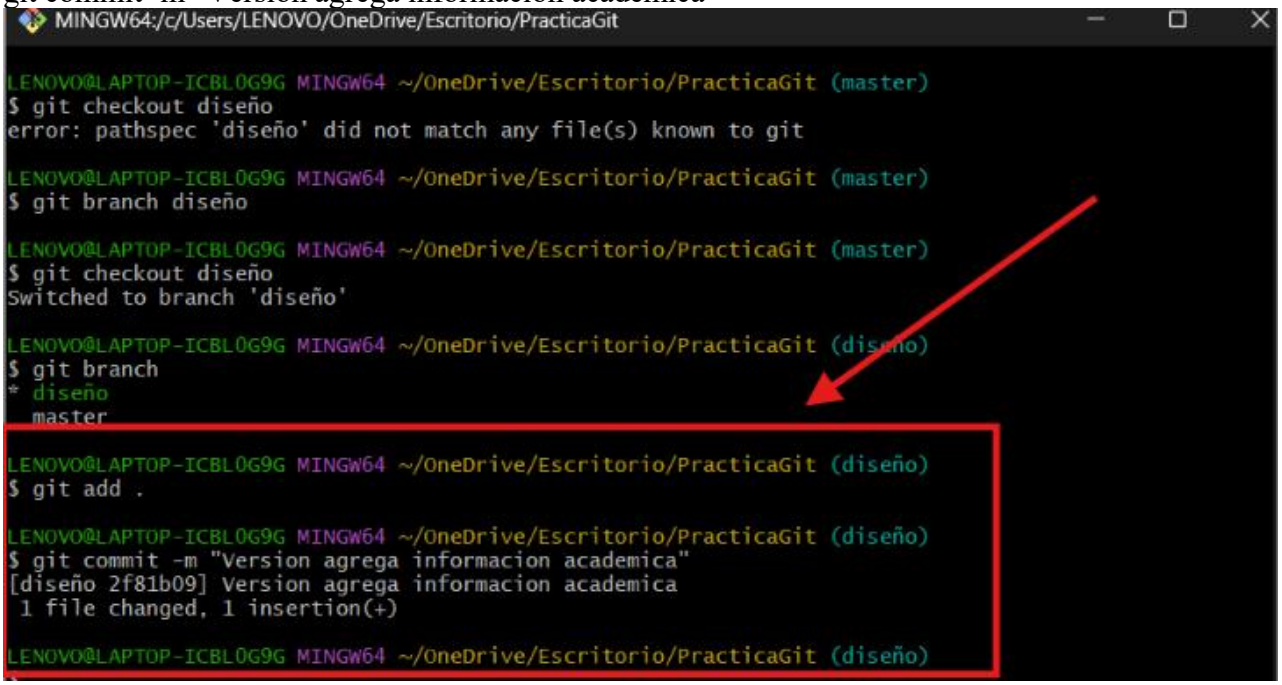


25. Modifique index.html agregando:

<h2>Mi información académica</h2>



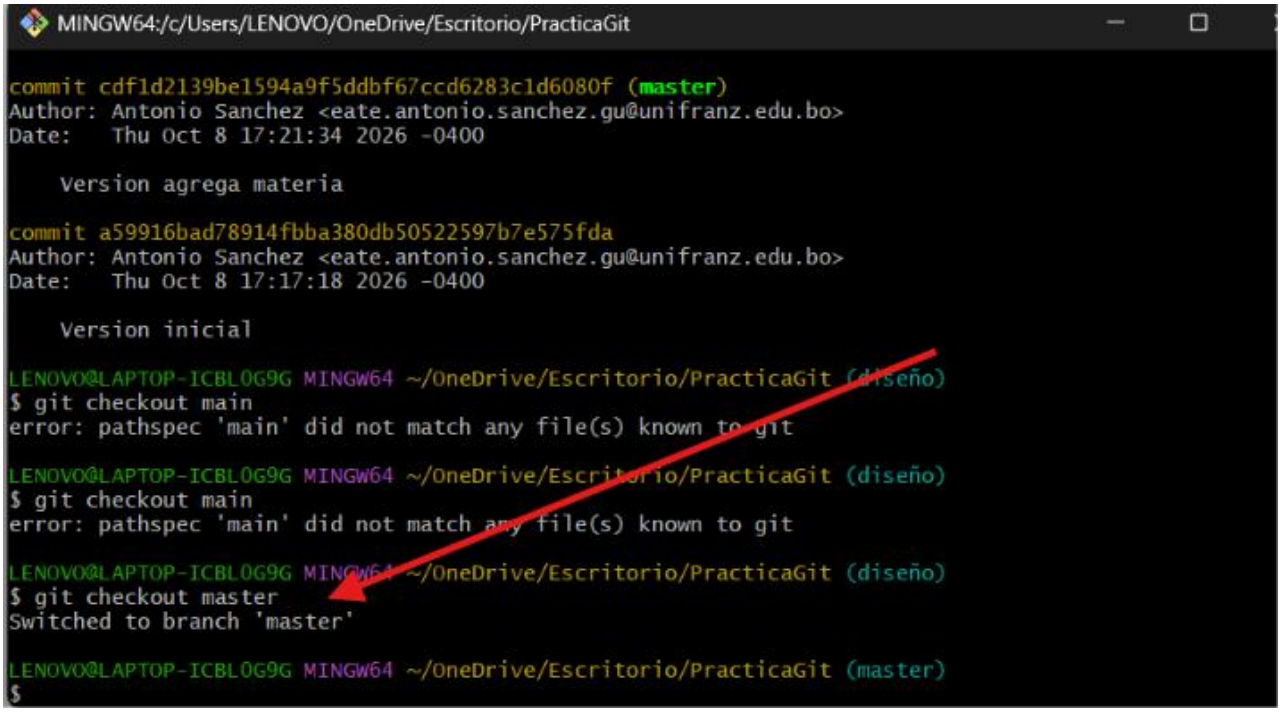
26. Realice:
    git add .
    git commit -m "Version agrega informacion academica"



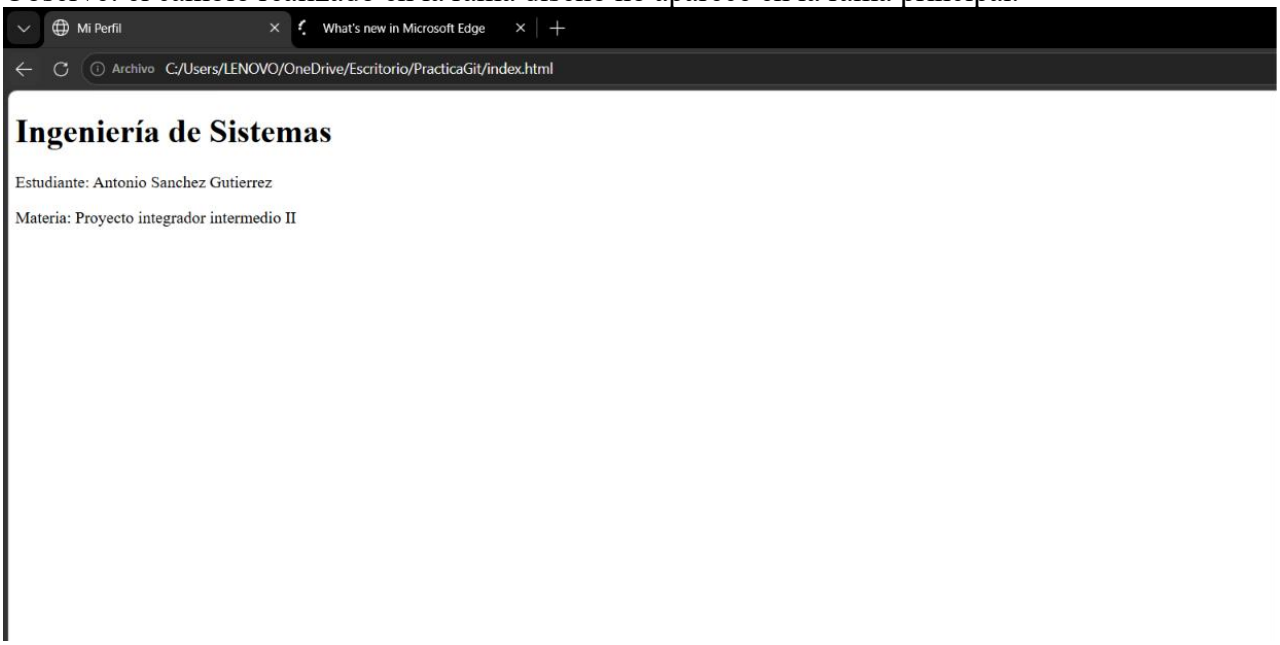
27. Consulte el historial.



28. Regrese a la rama principal: git
    checkout master
    Si Git creó la rama principal con el nombre main, utilice: git
    checkout main



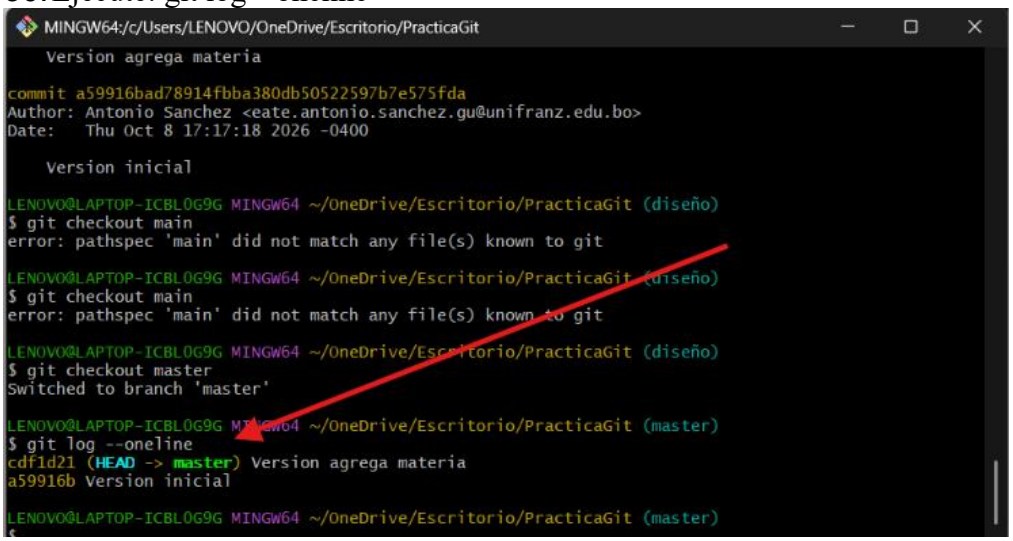
29. Abra index.html.
Observe: el cambio realizado en la rama diseño no aparece en la rama principal.



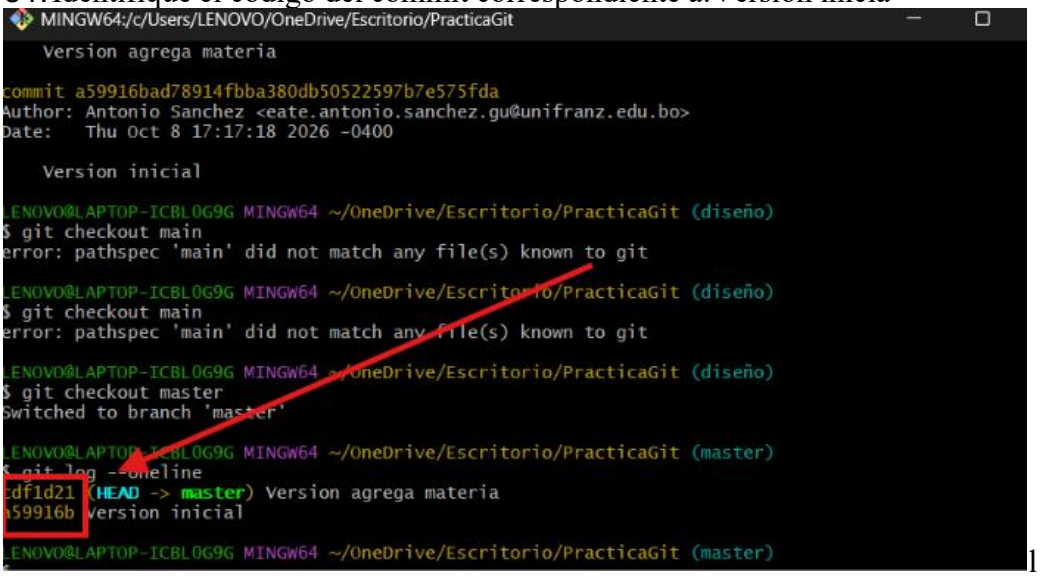
Pregunta:
¿Por qué la modificación realizada en diseño no aparece en master/main?
Porque cada rama es una línea de trabajo independiente. El commit “Version agrega informacion academica” se guardó solo en la rama diseño. La rama principal sigue apuntando al commit anterior (“Version agrega materia”), por eso Git, al cambiar de rama, te muestra los archivos tal como estaban en esa rama. Para que el cambio aparezca en la principal habría que unir las ramas con git merge.

F. Recuperar información del historial

30. Ejecute: git log --oneline



31. Identifique el código del commit correspondiente a:Version inicia



1

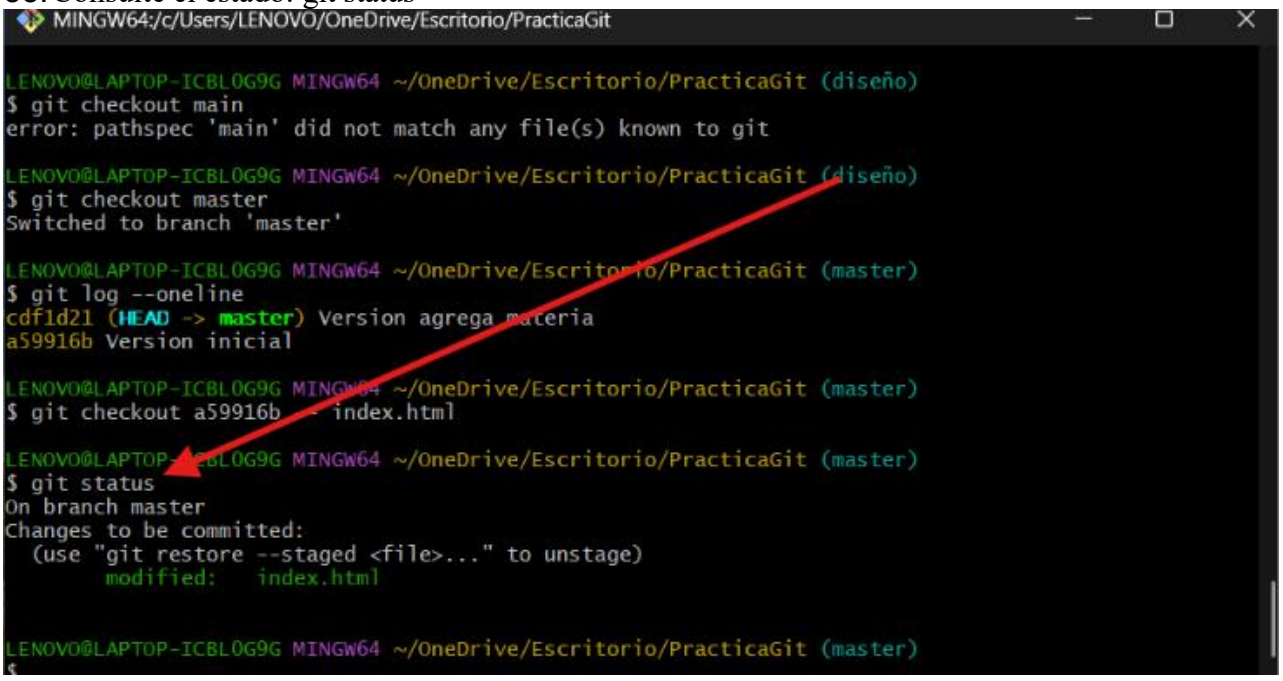
32. Use:

git checkout CODIGO_COMMIT -- index.html

Reemplace CODIGO_COMMIT por el código correspondiente.

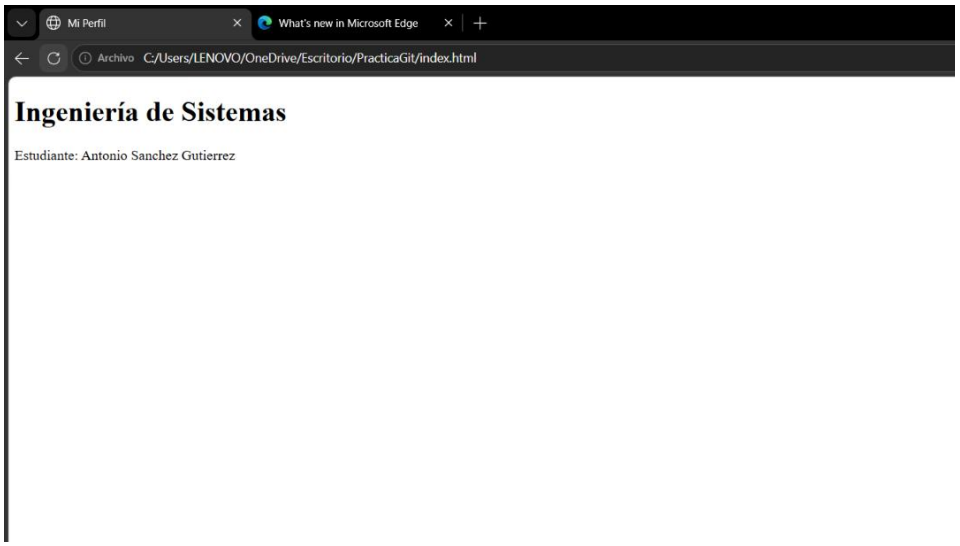


33. Consulte el estado: git status



34. Observe qué ocurrió con index.html.

Se modifico el index.html



Pregunta:

¿Qué ocurrió con index.html? Explique

Git tomó la versión del archivo guardada en el commit "Version inicial" y la copió sobre el archivo actual. El contenido volvió a como estaba al principio (sin la línea de la materia), y además el cambio quedó ya en el área de preparación (por eso git status lo muestra en verde). Todavía no es un commit nuevo; si lo quisieras guardar, tendrías que hacer git commit.

Preguntas finales. Responder de forma concreta:

1. **¿Cuál es la diferencia entre área de trabajo y staging area?**
   Es la carpeta donde editas tus archivos normalmente. El staging area (área de preparación) es una zona intermedia donde eliges qué cambios irán en el próximo commit, usando git add.

2. **¿Para qué sirve git log --oneline?**
   Para ver el historial de commits de forma resumida: una línea por commit, con su código corto y su mensaje.

3. **¿Qué diferencia existe entre git restore y git checkout?**
   git restore sirve solo para restaurar archivos (descartar cambios o sacarlos del staging).
   git checkout hace más cosas: cambia de rama y también puede traer un archivo de otro commit. Git creó restore y switch para separar esas funciones y que sea menos confuso

4. **¿Para qué sirve una rama (branch)?**
   Para trabajar en cambios o ideas nuevas de forma aislada, sin afectar la versión principal. Cuando el cambio está listo, se puede unir a la rama principal.

Presentar:
En 1 archivo PDF capturas de pantalla con los comandos ejecutados y las salidas obtenidas, respuestas a las preguntas, capturas de pantalla de la página **index.html** y preguntas finales.